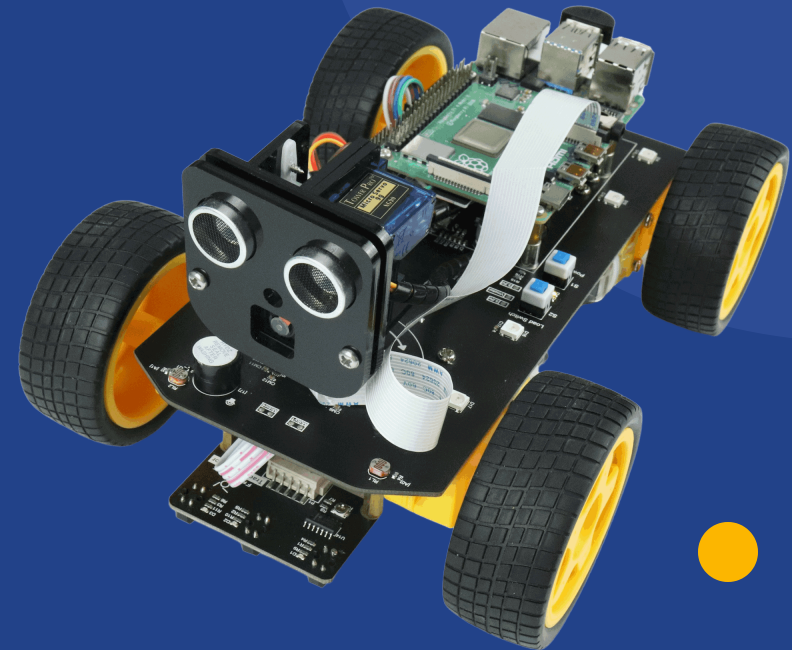




LESSON 7 – FACE TRACKING

Machine Learning with Raspberry Pi & Smart Car

The car can see — now it gets a **brain for vision**. It finds your face and turns its head to follow you.





LESSON 7 – FACE TRACKING

From eyes to a brain for vision

Lesson 6 gave the car **eyes**. Today it learns to **find a face** and **follow it** with its head.

- Lesson 6 gave the car **eyes** – a live video stream.
- Now we add a **brain for vision**: find a face and **follow it** with the head.
- Same camera stream – plus a **second connection** to send servo commands.



See the video → find the face



LESSON 7 – FACE TRACKING

Today's goal: one program that follows you

Build a program that sees the video, finds a face, and keeps it **centered**.

- 1 **Receive video** from the Pi (just like Lesson 6).
- 2 **Detect a face** with OpenCV.
- 3 Turn the **head (pan + tilt)** to keep that face centered.



Each function does **one job** – Lesson 8 will reuse them.

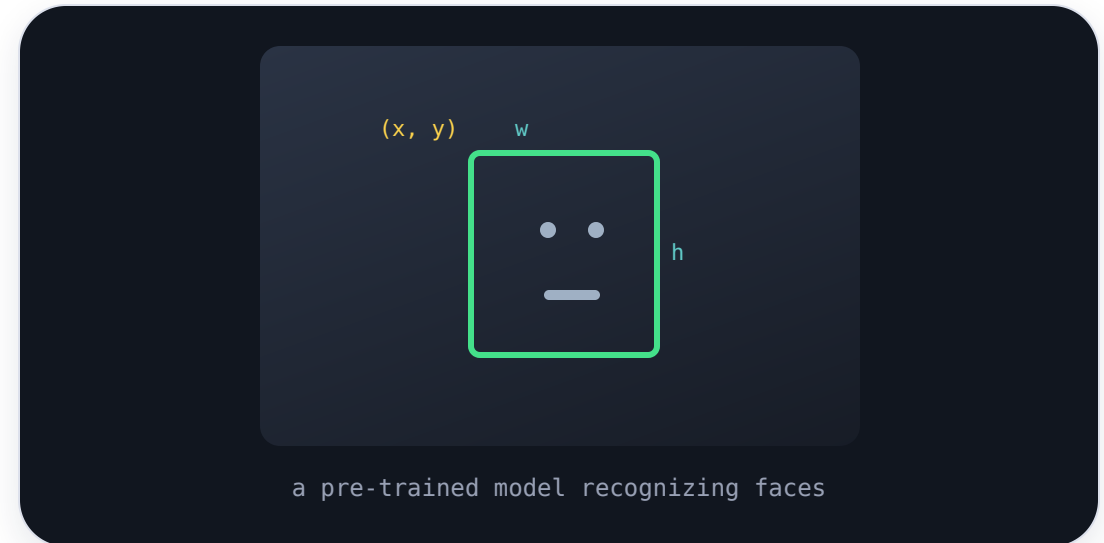


LESSON 7 – FACE TRACKING • THE MACHINE-LEARNING IDEA

How a computer "sees" a face

OpenCV uses a **cascade** – a file trained on what a face looks like.

- 1 Convert the picture to **grey**.
- 2 Scan it for **face-shaped patterns**.
- 3 Return a **box** `(x, y, w, h)` around each face.





LESSON 7 – FACE TRACKING • THE KEY IDEA

Two connections: video in, commands out

This program opens **two** connections to the Pi's full server (`main.py`).



Port 8000 – receive video (pictures *in*).



Port 5000 – send servo commands (orders *out*).

START THE SERVER FIRST

Lessons 7 & 8 talk to the **full** `main.py` server – both ports, not the Lesson 6 server.

• • • Pi terminal

```
cd Smart-Car-Level-3/Code/Server  
sudo python3 main.py
```

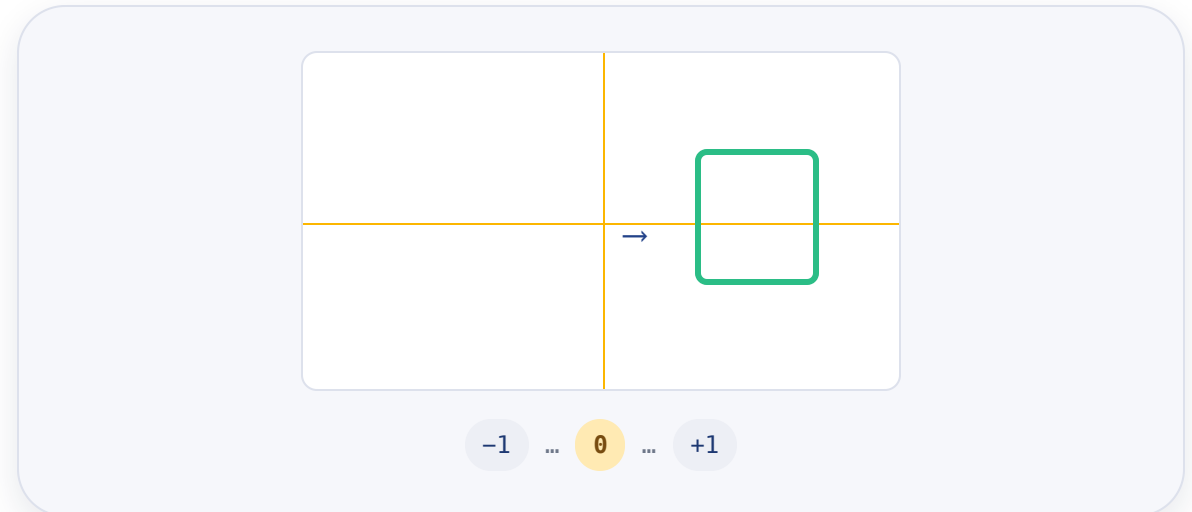


LESSON 7 – FACE TRACKING • THE KEY IDEA

Face position → head movement

Compare the face's center to the picture's center, then nudge the head toward it.

- Find the face's center, compare it to the **picture's center**.
- **Offset:** -1 (far left/top) ... 0 (centered) ... +1 (far right/bottom).
- Ignore tiny offsets (**dead zone**) so it doesn't jitter.
- Never move past the servo's **limits (clamp)**.





BUILD THE FILE • CHUNK 1 / 12

Start the file: imports

`cv2` finds faces; `COMMAND` gives us the command names — like Lesson 2.

- `car_setup` is back — so we can import `COMMAND`.
- `socket`, `struct`, `sys` — network + the IP you type.
- `cv2` shows pictures and finds faces; `numpy` decodes them.

```
lesson_7_face_tracking.py

"""Lesson 7: face detection and tracking."""

import car_setup    # add the car's code to the path
import socket       # connect to the Pi
import struct       # read the 4-byte picture size
import sys          # read the IP you type

import cv2          # OpenCV: shows + finds faces
import numpy as np  # bytes -> an image

from command import COMMAND # the server's command names
```



BUILD THE FILE • CHUNK 2 / 12

Settings

Two ports, picture size, and the head's **pan / tilt limits**.

- `8000` = video in • `5000` = commands out.
- `DEAD_ZONE` ignores tiny offsets (stops jitter).
- The **cascade file** lives in the car's `Client` folder.

```
lesson_7_face_tracking.py

# ---- settings ----
VIDEO_PORT = 8000      # receive pictures here
COMMAND_PORT = 5000    # send servo commands here
FRAME_WIDTH = 400
FRAME_HEIGHT = 300

PAN_SERVO = "0"        # left/right head servo
TILT_SERVO = "1"       # up/down head servo
PAN_MIN, PAN_MAX = 0, 180
TILT_MIN, TILT_MAX = 80, 180
SERVO_STEP = 4         # degrees per step
DEAD_ZONE = 0.15       # ignore tiny offsets

CASCADE_PATH = car_setup.CODE_DIR / "Client" \
               / "haarcascade_frontalface_default.xml"
```




BUILD THE FILE • CHUNK 3 / 12

Placeholders

The detector, both connections, and the head angles start **empty / centered**.

- One spot each for the detector and both connections.
- `pan_angle` / `tilt_angle` track where the head is **now**.
- `90` = straight ahead – we nudge from there.

```
lesson_7_face_tracking.py
```

```
# ---- things created in setup ----
cmd = None           # command names like cmd.CMD_SERVO
face_cascade = None  # the face detector
video_socket = None  # receives pictures
video_file = None    # that connection, read like a file
command_socket = None # sends servo commands
pan_angle = 90.0     # current left/right angle
tilt_angle = 90.0    # current up/down angle
```



BUILD THE FILE • CHUNK 4 / 12

setup(): load the detector & connect

Load the face cascade (and check it loaded), then open **both** connections.

- `empty()` is `True` if the file failed to load.
- Open the **video** connection (8000)...
- ...and the **command** connection (5000). Two `connect()` calls.

```
lesson_7_face_tracking.py

def setup():
    global cmd, face_cascade, video_socket, video_file, command_socket
    if len(sys.argv) < 2:
        raise SystemExit("Pass the Pi's IP, e.g. python this.py 192.168.1.50")

    cmd = COMMAND()
    face_cascade = cv2.CascadeClassifier(str(CASCADE_PATH))
    if face_cascade.empty():
        raise FileNotFoundError("Could not load the face file: " + str(CASCADE_PATH))

    pi_ip = sys.argv[1]
    video_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    video_socket.connect((pi_ip, VIDEO_PORT)) # pictures in
    video_file = video_socket.makefile("rb")

    command_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    command_socket.connect((pi_ip, COMMAND_PORT)) # commands out
```



BUILD THE FILE • CHUNK 5 / 12

read_frame(): the same trick as Lesson 6

Read the 4 size-bytes, then the JPEG, then decode it – copied straight from Lesson 6.

- Read the **4 size-bytes**; turn them into a number.
- If the size is `0` → the stream ended.
- Reusing a known idea – recognizing it is a real skill.

```
lesson_7_face_tracking.py

def read_frame():
    """Read one picture, or None when the stream ends."""
    header = video_file.read(4)
    if len(header) != 4:
        return None
    length = struct.unpack("<L", header)[0]
    if length == 0:
        return None
    jpeg = video_file.read(length)
    if len(jpeg) != length:
        return None
    return cv2.imdecode(np.frombuffer(jpeg, np.uint8), cv2.IMREAD_COLOR)
```



BUILD THE FILE • CHUNKS 6-7 / 12

detect_faces() & biggest_face()

Grey the picture and find every face — then pick the **largest** box (the closest person).

- `detectMultiScale` returns a box per face.
- Biggest box $\approx$ **closest face** (`w × h` is its area).
- Lesson 8 will improve on this assumption.

```
lesson_7_face_tracking.py

def detect_faces(frame):
    """Return a list of (x, y, w, h) boxes, one per face."""
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    # 1.3 and 5 control how carefully it searches.
    return face_cascade.detectMultiScale(gray, 1.3, 5)

def biggest_face(faces):
    """Return the largest face's box, or None if there are none."""
    if len(faces) == 0:
        return None
    return max(faces, key=lambda face: face[2] * face[3])
```



BUILD THE FILE • CHUNK 8 / 12

send_servo(): build a CMD_SERVO command

Reuse the command idea from **Lesson 2** – now sent over port 5000.

- Format: `CMD_SERVO#<channel>#<angle>` + a newline.
- Text must be sent as **bytes** – `.encode("utf-8")`.
- The same `#`-joined strings from Lesson 2.

```
lesson_7_face_tracking.py

def send_servo(channel, angle):
    """Send one command, e.g. CMD_SERVO#0#95."""
    message = (cmd.CMD_SERVO + "#" + channel
               + "#" + str(int(angle)) + "\n")
    command_socket.send(message.encode("utf-8"))
```



BUILD THE FILE • CHUNKS 9-10 / 12

track_face() & clamp(): nudge the head

Turn position into an **offset**, skip tiny offsets, then nudge pan / tilt within limits.

- **Dead zone:** already centered → don't move.
- Nudge pan / tilt toward the face.
- `clamp()` keeps angles inside the servo limits.

```
lesson_7_face_tracking.py

def track_face(face):
    global pan_angle, tilt_angle
    if face is None:
        return
    x, y, w, h = face
    center_x = x + w / 2
    center_y = y + h / 2
    # offset: -1 at left/top, +1 at right/bottom, 0 in the middle.
    offset_x = (center_x / FRAME_WIDTH - 0.5) * 2
    offset_y = (center_y / FRAME_HEIGHT - 0.5) * 2
    if abs(offset_x) < DEAD_ZONE and abs(offset_y) < DEAD_ZONE:
        return # already centered
    pan_angle = clamp(pan_angle + SERVO_STEP * offset_x, PAN_MIN, PAN_MAX)
    tilt_angle = clamp(tilt_angle - SERVO_STEP * offset_y, TILT_MIN, TILT_MAX)
    send_servo(PAN_SERVO, pan_angle)
    send_servo(TILT_SERVO, tilt_angle)

def clamp(value, low, high):
    return max(low, min(high, value))
```



BUILD THE FILE • CHUNK 11 / 12

loop(): find, draw, follow

Read a frame → find the biggest face → draw a green box → move the head → show it.

- This reads top-to-bottom like **English**.
- `rectangle` draws the green box around the face.
- Press `q` to quit.

```
lesson_7_face_tracking.py

def loop():
    """Read pictures, find the face, follow it, show the video."""
    while True:
        frame = read_frame()
        if frame is None:
            break
        face = biggest_face(detect_faces(frame))
        if face is not None:
            x, y, w, h = face
            cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)
            track_face(face)
        cv2.imshow(WINDOW_NAME, frame)
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
```



BUILD THE FILE • CHUNK 12 / 12

Clean up + the main block

Close **both** connections and the window – then the usual safe wrapper.

- Two connections now → **two closes**.
- `destroyAllWindows()` shuts the OpenCV window.
- `finally` always cleans up – same habit as always.

```
lesson_7_face_tracking.py

def destroy():
    """Close both connections and the window."""
    if video_file is not None:
        video_file.close()
    if video_socket is not None:
        video_socket.close()
    if command_socket is not None:
        command_socket.close()
    cv2.destroyAllWindows()

if __name__ == "__main__":
    try:
        setup(); loop()
    except KeyboardInterrupt:
        pass
    finally:
        destroy()
```




LESSON 7 – FACE TRACKING

We built the whole file

Every function does **one job** – ready for Lesson 8 to reuse.

**lesson_7_face_tracking.py**

imports → settings → placeholders → setup() → read_frame() → detect_faces() → biggest_face() → send_servo() →
track_face() → clamp() → loop() → destroy() → main



Single-purpose functions pay off next lesson: Lesson 8 won't rewrite any of this – it will `import` the file and reuse these functions.



LESSON 7 – FACE TRACKING

Run it

Server first, then the tracker. The tracker needs the Pi's IP address.

Step 1 – on the Pi, start the car's server

```
Pi terminal

cd Smart-Car-Level-3/Code/Server
sudo python3 main.py
```

Step 2 – on the Pi or your laptop, run the tracker

```
terminal

cd Smart-Car-Level-3/Code/User
python lesson_7_face_tracking.py 192.168.1.50
```



Detection is CPU-heavy – a **laptop** runs much smoother than the Pi. Use `127.0.0.1` on the Pi.



A window shows the camera; the head follows your face. Press `q` to quit.



LESSON 7 – FACE TRACKING

Try it yourself

Feel the trade-off: responsiveness vs. jitter. There's no single "right" value.

- Change `SERVO_STEP` – bigger = faster but **jumper**.
- Change `DEAD_ZONE` – smaller = more sensitive (and twitchier).
- Change the **box color** in `loop()`.

CHALLENGE

Look further up and down

Widen `TILT_MIN` / `TILT_MAX` and see how far up and down the head can follow.



LESSON 7 – FACE TRACKING

Common issues

If tracking misbehaves, it's almost always one of these.



Could not load the face file

The cascade path is wrong – confirm it's in `Code/Client/`.



No green box

More light on your face; face the camera straight on.



Head jitters / overshoots

Raise `DEAD_ZONE` or lower `SERVO_STEP`.



Connection refused

Start `sudo python3 main.py` on the Pi first; check the IP.



Lighting wins

Lighting is the biggest factor in detection quality. Aim a lamp at your face if the box won't appear.



LESSON 7 – FACE TRACKING

The car follows your face!

You combined *seeing* with *moving* – vision in, servo commands out.

- ✓ Used **OpenCV** to detect a face in the video
- ✓ Steered the **head** to keep that face centered
- ✓ Combined **video in** (8000) with **commands out** (5000)



UP NEXT

Lesson 8 Multiple faces

Handle **multiple faces** and lock onto one – by **reusing this file**. We won't rewrite it; we'll

```
import it.
```



• COACH ONLY • HIDDEN SLIDE

End-of-class checklist

Don't present this slide. Run through it right after class.

1

Post the after-class recap

Summarise what you covered on the class page so students can catch up.

2

Log absent students

Post anyone who missed class in the **#attendance** Discord channel.

3

Charge the cars for Lesson 8

Lesson 8 handles multiple faces and **imports this file** – keep today's working copy.

EVERY CLASS

✓ Post a class announcement

✓ Introductions if needed

✓ Note absences in #attendance